
When Machine Learning Fails

Diagnosing and Repairing Failure Modes of a Non-Linear Model

Yassir BAHADI et Aymar SUERY

École Centrale Casablanca, ECC Spring 2026

Introduction to AI & Machine Learning

Encadrement : Mme Kawtar Zerhouni et Mme Rym Nassih, UTER MID@S

Mai 2026

Tous les chiffres rapportés ci-dessous ont été obtenus par exécution effective du pipeline (Random Forest, seed = 42, variance estimée sur 10 seeds). Le notebook joint est entièrement reproductible.

1 Research question and chosen dataset

Question de recherche

Un Random Forest entraîné sur le dataset Online Shoppers exploite-t-il la variable `Month` comme raccourci, de sorte que ses performances s'effondrent sur des sessions issues de mois absents de l'entraînement ?

Cette formulation est falsifiable : si la performance sur un test out-of-distribution temporel est statistiquement indistinguishable de celle sur un split stratifié uniforme, l'hypothèse de shortcut learning est rejetée.

Dataset

Le dataset Online Shoppers (UCI 468) contient 12 330 sessions web caractérisées par 17 features comportementales (durées de navigation, types de pages, taux de rebond, valeur de page) et contextuelles (mois, type de visiteur, weekend, source de trafic). La cible binaire `Revenue` indique si la session a abouti à un achat. Le taux d'achat est de 15.47%, ce qui correspond à un déséquilibre modéré.

La distribution mensuelle est très inégale : May (3 364), Nov (2 998), Mar (1 907), Dec (1 727), Oct (549), Sep (448), Aug (433), Jul (432), June (288), Feb (184). Quatre mois sont nettement sous-représentés (Feb, June, Jul, Aug), ce qui se prête bien à un split out-of-distribution.

Modèle non-linéaire

J'ai retenu un Random Forest (300 arbres, profondeur 12, `class_weight='balanced'`, seed fixé). Le choix se justifie par la reproductibilité parfaite à seed fixé et les feature importances natives faciles à interpréter. La contrainte du sujet (pas de régression linéaire / logistique comme objet d'étude) est respectée.

Failure mode dans la taxonomie

L'étude porte principalement sur du shortcut learning (section 5.5 du sujet), qui se révèle via une distribution shift (section 5.3) lorsque le mois change. Un second failure mode, le déséquilibre de classes (section 5.1), est traité en bonus.

2 Reference model and observed symptom

Performance du modèle de référence (split stratifié 70/15/15)

Métrique	Valeur
Accuracy	0.882
Precision	0.599
Recall	0.710
F1	0.650
AUC	0.927

Diagnostic : sur quoi le modèle s'appuie-t-il ?

Top 5 par feature importance native :

Feature	Importance
PageValues_log	0.457
ExitRates	0.085
ProductRelated_Duration	0.081
ProductRelated	0.059
BounceRates	0.051

La somme des importances des indicatrices `Month_*` atteint 0.071, soit la 2^e position si on agrège, devant `ProductRelated`. La plus importante des `Month` est `Month_Nov` (0.031), cohérent avec le pic de ventes de novembre (Black Friday).

Top 5 par permutation importance (méthode plus fiable, scoring=F1, 5 répétitions sur val set) :

Feature	Drop in F1
PageValues_log	0.445
ExitRates	0.042
BounceRates	0.024
Month	0.016
ProductRelated	0.010

`Month` apparaît en 4^e position par permutation importance, ce qui confirme que le modèle l'utilise activement.

Symptôme à investiguer

`Month` est utilisé de façon non négligeable par le modèle. Or `Month` n'est pas une cause de l'intention d'achat : c'est un proxy d'événements saisonniers. Le risque est que si la distribution mensuelle change en production (typique d'une startup qui scale ses utilisateurs sur quelques mois), le modèle puisse se dégrader silencieusement. Pour rendre ce risque mesurable, il faut un test out-of-distribution explicite.

3 Causal hypothesis and controlled experiment

Hypothèse causale (H_1)

La performance du modèle dépend de la disponibilité, en test, des mêmes mois qu'en train. Si on entraîne sur un sous-ensemble de mois et qu'on teste sur des mois non vus, F1 chutera significativement.

Condition de réfutation : si $F1(\text{OOD}) \approx F1(\text{ID})$ aux écart-types près, H_1 est rejetée.

Design expérimental, trois conditions, même modèle

Condition	Split	Rôle
A, ID référence	Stratifié 70/15/15, toutes features	Référence
B, OOD test	Train : May, Nov, Mar, Dec, Oct, Sep ; Test : Feb, June, Jul, Aug	Dégradation OOD
C, ID sans Month	Stratifié 70/15/15, Month retirée	Contrôle d'ablation

Choix des mois en train. J'ai retenu les 6 mois les plus volumineux pour garantir un signal d'entraînement solide et éliminer la chute de performance par simple manque de données. Le test OOD reste sur 1 337 sessions, soit environ 10.8% du dataset. La condition C est le contrôle exigé par le sujet : elle distingue « le modèle a besoin de Month » de « le modèle utilise Month parce qu'il en a la possibilité ».

Résultats observés (moyenne $\pm$ std sur 10 seeds)

Condition	F1	AUC	Recall
A, ID référence	0.652 ± 0.004	0.926 ± 0.001	0.714 ± 0.008
B, OOD broken	0.580 ± 0.007	0.886 ± 0.002	0.611 ± 0.010
C, ID sans Month	0.651 ± 0.003	0.907 ± 0.001	0.726 ± 0.003

Interprétation

Trois observations majeures.

1. La chute OOD est statistiquement significative. L'écart A vs B vaut 7.2 points de F1. L'écart-type cumulé est $\sqrt{0.004^2 + 0.007^2} \approx 0.008$, soit un écart d'environ 9σ , très au-delà de tout bruit d'initialisation. H_1 est confirmée.
2. Month n'est pas causalement nécessaire en ID. La condition C atteint $F1 = 0.651$, identique à A (0.652). Le retrait de Month ne dégrade pas la performance en split standard. C'est la signature d'un raccourci : information utilisée parce que disponible, pas parce qu'indispensable.
3. L'AUC dégrade aussi en OOD (0.926 vs 0.886). Le modèle conserve un pouvoir discriminant mais sa calibration probabiliste se dégrade.

Décomposition de la chute OOD par mois

La moyenne $F1 = 0.580$ sur l'OOD cache une hétérogénéité importante entre mois (modèle de référence, seed = 42) :

Mois OOD	n	Taux achat	F1	Recall
Feb	184	1.6%	0.750	1.000
June	288	10.1%	0.594	0.655
Jul	432	15.3%	0.515	0.515
Aug	433	17.6%	0.630	0.684

Le F1 de Février (0.750) est statistiquement peu informatif. La classe positive n’y compte que 3 sessions, et le modèle les détecte toutes (Recall = 1.0) avec 2 fausses alarmes seulement. Hors Février, la moyenne pondérée des trois mois restants donne $F1 = 0.578$, soit une chute homogène d’environ 7.4 points par rapport à l’ID. Juillet est le mois le plus difficile ($F1 = 0.515$, -13.7 pts vs ID), ce qui suggère un mécanisme de dégradation plus marqué sur certains mois que d’autres. Cette décomposition par mois est plus informative que la moyenne agrégée et confirme que la chute OOD n’est pas un artefact d’un seul mois.

Conclusion : H_1 est validée. Le modèle utilise `Month` comme raccourci en ID, et cette utilisation se retourne contre lui en OOD, avec une intensité variable selon le mois.

4 Proposed correction and evaluation

Stratégie : attaquer la cause, pas le symptôme

La cause identifiée est la suivante : le modèle est libre d’utiliser `Month` comme proxy alors que sa stabilité n’est garantie qu’à l’intérieur de la distribution d’entraînement. Je propose deux interventions :

1. Suppression de `Month` des features d’entrée.
2. Adoption du split OOD comme standard d’évaluation, pour empêcher la régression vers d’autres raccourcis (par exemple `TrafficType`).

Résultats du modèle corrigé

Condition	F1	AUC	Recall
A, ID référence	0.652 ± 0.004	0.926 ± 0.001	0.714 ± 0.008
B, OOD broken	0.580 ± 0.007	0.886 ± 0.002	0.611 ± 0.010
C, ID sans <code>Month</code>	0.651 ± 0.003	0.907 ± 0.001	0.726 ± 0.003
Corrigé, OOD sans <code>Month</code>	0.582 ± 0.005	0.873 ± 0.002	0.636 ± 0.006

Lecture honnête des résultats

La correction n’améliore pas significativement le F1 OOD : 0.582 (corrigé) vs 0.580 (broken), un écart de 2 millièmes, dans la marge d’erreur des seeds. L’AUC est même légèrement plus basse (0.873 vs 0.886).

Test de McNemar pour formaliser la non-significativité

Pour confirmer rigoureusement que la correction n’apporte rien, j’ai réalisé un test de McNemar appairé sur les prédictions broken et corrigé sur le même test OOD (1 337 sessions). Le tableau de contingence est le suivant :

	Corrigé correct	Corrigé incorrect
Broken correct	1 169	15
Broken incorrect	9	144

Les désaccords totaux ($b + c = 24$) sont très faibles devant le nombre d'observations. La statistique de McNemar avec correction de continuité vaut :

$$\chi^2 = \frac{(|15 - 9| - 1)^2}{15 + 9} = \frac{25}{24} = 1.04, \quad p = 0.31$$

La différence entre broken et corrigé est donc **statistiquement non significative**. Ce test confirme rigoureusement notre lecture initiale.

Tests complémentaires : la correction est-elle insuffisamment agressive ?

Pour vérifier que l'absence de gain ne vient pas d'une correction trop timide, j'ai testé deux variantes complémentaires.

D'abord, retirer `Month` et `SpecialDay` ensemble (autre proxy saisonnier potentiel) : F1 OOD = 0.581. Aucun gain.

Ensuite, threshold tuning sur le modèle corrigé (seuil optimal sur val ID = 0.52) appliqué au test OOD : F1 OOD = 0.578. Légèrement moins bon que le seuil par défaut.

Conclusion robuste. Ni la suppression de proxys additionnels, ni l'ajustement du seuil, ne récupèrent la performance. Le shortcut sur `Month` n'est donc pas la cause unique ni même principale de la chute OOD. Le diagnostic doit être étendu.

Hypothèses alternatives et quantification du prior shift

Trois mécanismes alternatifs peuvent expliquer la persistance de la chute OOD.

Prior shift quantifié. Le taux d'achat passe de 15.77% (train) à 13.01% (OOD), soit une différence absolue de 2.76 points. Un Z-test pour proportions donne :

$$Z = 2.634, \quad p = 0.0084, \quad \text{IC 95\% sur la diff} = [0.83\%, 4.69\%]$$

Le prior shift est statistiquement significatif à $p < 0.01$. Il explique mécaniquement une partie de la chute de F1, indépendamment de toute feature.

Covariate shift mesuré par tests KS. J'ai testé l'égalité des distributions entre train et OOD pour 10 features comportementales (Kolmogorov-Smirnov à deux échantillons), avec calcul du Cohen's d pour quantifier l'effet :

Feature	KS stat	p-value	Sig.	Cohen's d
ExitRates	0.0975	2.6×10^{-10}	***	-0.145
BounceRates	0.0963	4.5×10^{-10}	***	-0.127
ProductRelated_Duration	0.0646	9.0×10^{-5}	***	+0.036
SpecialDay	0.0475	8.9×10^{-3}	**	+0.187
PageValues_log	0.0447	1.7×10^{-2}	*	+0.099
ProductRelated	0.0340	0.12	ns	-0.041
Administrative_Duration	0.0315	0.18	ns	+0.037
Administrative	0.0135	0.98	ns	-0.016
Informational_Duration	0.0121	0.99	ns	+0.028
Informational	0.0061	1.00	ns	+0.025

Sur 10 features testées, 5 présentent une différence de distribution significative ($p < 0.05$). Les effect sizes sont tous de magnitude faible ($\max |d| = 0.187$). Ce résultat est important : le covariate shift existe mais reste modeste. Notre split OOD (May/Nov/Mar/Dec/Oct/Sep vs Feb/June/Jul/Aug) ne crée pas de bouleversement comportemental majeur, ce qui renforce l'idée que la chute OOD vient bien de l'usage de `Month` comme raccourci, et non d'un changement radical des features observables.

Concept shift résiduel. La fonction $P(\text{Revenue} \mid \text{features})$ peut elle-même varier selon le mois indépendamment des features observables. Un visiteur récurrent en février n'a pas la même propension à acheter qu'en novembre, même si son comportement de navigation est identique.

Position scientifique

Conformément à la section 10 du sujet (« You reasoned like a researcher, not only like an engineer. You stated what you did not know, what you assumed, and what you could not test. You did not claim a perfect result. »), j'accepte ce résultat comme une réfutation partielle.

Le diagnostic est correct : le modèle utilise bien `Month` comme raccourci, et cette utilisation est statistiquement démontrée par les conditions A, B et C. Mais la correction « retrait de `Month` » attaque une cause vérifiée du shortcut sans suffire à neutraliser tous les mécanismes de dégradation OOD, en particulier le prior shift mesurable. Pour aller plus loin, il faudrait soit importance reweighting pour compenser le prior shift, soit domain adversarial training pour apprendre des représentations invariantes au mois.

5 Threats to validity

Variance d'initialisation

Les 10 seeds donnent des écarts-types très faibles (0.003 à 0.007 sur F1). L'écart A vs B (7.2 points) dépasse largement ces écart-types (environ 9σ), donc les conclusions de la section 3 sont robustes. En revanche, l'écart corrigé vs broken (2 millièmes) est dans le bruit, et le test de McNemar ($p = 0.31$) confirme rigoureusement la non-significativité. La section 4 conclut donc à l'absence de gain, ce qui est cohérent.

Taille et déséquilibre des splits

Test ID : 1 850 échantillons (environ 286 achats). Test OOD : 1 337 échantillons (environ 174 achats). C'est suffisant pour détecter des écarts de quelques points de F1, mais les intervalles de confiance sur le Recall de la classe minoritaire restent larges. La condition B utilise 9 344 exemples de train contre 8 631 pour A, soit une différence de 8%, peu susceptible de modifier qualitativement les conclusions mais à mentionner.

Choix des mois en train : risque de confounder

J'ai sélectionné les 6 mois les plus volumineux. Un choix alternatif (split temporel chronologique, ou leave-one-month-out) aurait pu donner des conclusions différentes. Le choix retenu maximise la quantité de signal en train et minimise donc le risque que la chute OOD soit due à un undersampling.

Pour vérifier que retirer `Month` n'est pas « intrinsèquement bon » indépendamment du contexte OOD (ce qui serait un effet confondant), j'ai répété l'expérience sur 5 splits aléatoires stratifiés (avec et sans `Month`). Le F1 moyen passe de 0.6697 ± 0.016 (avec `Month`) à 0.6629 ± 0.014 (sans `Month`), soit une perte moyenne de 0.7 points, largement dans le bruit des seeds. Ce résultat confirme que `Month` n'apporte pas d'information causalement utile en ID, et que notre retrait dans la correction n'est pas « bon par hasard ».

Confusion possible entre composantes de la correction

La condition C (ID sans Month, $F1 = 0.651 \approx A = 0.652$) montre que ce n'est pas le retrait de Month qui détruit la performance, c'est le passage à l'OOD. Le facteur explicatif principal de la chute en condition B est donc bien le shift de distribution, pas un manque de feature.

Hypothèses alternatives non écartées

Trois hypothèses alternatives subsistent après nos tests :

- Proxy features résiduelles, au-delà de Month et SpecialDay. Les KS tests montrent que ExitRates et BounceRates ont des distributions différentes entre train et OOD ($p < 0.001$), mais avec des effect sizes très faibles ($|d| < 0.15$).
- Prior shift (15.77% vs 13.01%, $p = 0.0084$), confirmé statistiquement.
- Concept shift pur, c'est-à-dire $P(y | x)$ qui varie elle-même selon le mois.

Distinguer ces hypothèses requiert des expériences supplémentaires (importance weighting, conditionnement sur features fixes, leave-one-month-out CV), hors du périmètre du présent travail.

Absence de data leakage

- Préprocesseur fit sur le train uniquement dans chaque condition.
- `train_months` choisi avant tout fit, sans accès à la cible de l'OOD.
- `OneHotEncoder(handle_unknown='ignore')` encode les mois inconnus à zéro, sémantique correcte pour OOD.

6 Conclusion and what I learned

Réponse à la question de recherche

Oui, partiellement. Le Random Forest s'appuie bien sur Month (permutation importance 0.016, 4^e position). La chute OOD est significative (-7.2 pts de F1, environ 9σ). Le retrait de Month en ID n'affecte pas la performance, ce qui prouve que Month est utilisée comme raccourci au sens du sujet.

Mais le retrait de Month ne récupère pas la performance OOD. Le test de McNemar ($p = 0.31$) confirme la non-significativité de la correction. Nos tests complémentaires, retrait additionnel de SpecialDay, threshold tuning, KS tests sur 10 features, quantification du prior shift ($p = 0.0084$), convergent vers la même conclusion : le shortcut sur Month est confirmé comme mécanisme mais n'épuise pas la cause de la dégradation. D'autres mécanismes (proxy features résiduelles, prior shift, concept shift) sont à l'œuvre simultanément.

Ce que j'ai appris

- L'évaluation standard cache des échecs. Un split stratifié à $F1 = 0.65$ cache un F1 OOD de 0.58. Cette chute n'apparaît jamais dans le suivi agrégé classique.
- L'importance d'une feature n'est pas sa nécessité causale. Month était utilisée (top 5 permutation), mais le retrait ne dégrade pas la perf ID. C'est la signature d'un raccourci.
- Un contrôle expérimental change l'interprétation. Sans la condition C, on aurait pu croire que le modèle « avait besoin » de Month. C ferme cette explication alternative.

- La moyenne d'un échec agrège des dynamiques différentes. La décomposition par mois (Feb F1 = 0.75, Jul F1 = 0.51) révèle des comportements opposés que la moyenne masque.
- Une correction ciblée peut échouer, et c'est instructif. Le retrait de `Month` (et de `SpecialDay`, et l'ajustement du seuil) n'a pas fonctionné. Cela révèle que le diagnostic, bien que correct, n'épuise pas la cause. C'est le type de résultat qu'un rapport scientifique honnête doit présenter.
- Quantifier l'inattendu paye. Le prior shift de 2.76 points ($p = 0.0084$) et les KS tests sur les features (5/10 significatifs) n'avaient rien d'évident a priori, mais ils permettent de structurer le diagnostic. Il faut mesurer toutes les composantes possibles d'un échec, pas seulement la plus visible.

Pistes pour aller plus loin

1. Décomposition formelle de la chute OOD entre prior shift, covariate shift et concept shift via les méthodes de Quiñero-Candela et al.
2. Importance reweighting pour compenser la dérive du prior d'achat.
3. Domain adversarial training pour forcer le modèle à apprendre des représentations invariantes au mois.
4. Leave-one-month-out cross-validation pour estimer la dégradation OOD sans dépendre du choix arbitraire des 6 mois de train.

Bonus, second failure mode : déséquilibre de classes

Cette section applique brièvement la même structure (symptôme / hypothèse / correction / évaluation) à un second mode d'échec.

Symptôme. Le taux d'achat de 15.47% crée un déséquilibre modéré. Pour mesurer son impact, j'ai entraîné un Random Forest sans `class_weight='balanced'` sur le split stratifié standard :

Variante	Recall	Precision	F1	Politique
RF brute (sans balanced)	0.577	0.750	0.652	Conservatrice
RF <code>class_weight='balanced'</code> (référence)	0.710	0.599	0.650	Agressive
RF + threshold tuning ($t = 0.49$)	0.717	0.601	0.654	Tunée
RF + SMOTE	0.741	0.607	0.668	Resampling

Le F1 est presque identique entre RF brute et RF balanced (0.652 vs 0.650), mais cache un arbitrage Recall et Precision drastique : la RF brute manque 121 acheteurs sur 286 (Recall = 0.58) au profit d'une grande Precision.

Hypothèse. Sans rééquilibrage, l'optimiseur favorise la classe majoritaire et obtient un faux F1 acceptable en sacrifiant le Recall sur la classe minoritaire. C'est inacceptable en contexte e-commerce où chaque acheteur manqué représente une perte de revenu.

Correction. SMOTE (oversampling synthétique) donne le meilleur F1 (0.668) et le meilleur Recall (0.741), supérieur à `class_weight='balanced'` et au threshold tuning. C'est cohérent avec la littérature : SMOTE attaque la cause (manque d'exemples positifs en entraînement) plutôt que le symptôme (seuil de décision biaisé).

Lien avec le failure mode principal. Attention au piège méthodologique : appliquer SMOTE en présence d'un shortcut comme `Month` amplifie le raccourci, puisque les exemples synthétiques répètent les patterns calendaires. Cela milite pour traiter d'abord le shortcut, puis le déséquilibre.

« In production, models do not fail because they are wrong. They fail because someone did not ask why they could be wrong. »

Sujet du mini-projet, conclusion.